Code-as-Steering: Continual Programmatic Adaptation of a Frozen Vision-Language-Action Model

ASPIRE project — working document

October 26, 2026

Abstract

A coding agent builds a library of executable programs that change how a *frozen* vision-language-action model (VLA) behaves at inference time. A program reads the observation and, crucially, the VLA's own internal state (activations, attention), and decides which internal interventions to apply: additive steering directions in chosen layers, ablations, attention edits, or, as a last resort, small adapters. The agent discovers these interventions by controlled experiments in simulation, verifies them on held-out situations, and stores the verified ones with their applicability conditions so they can be reused and composed on new tasks. The VLA weights never change, so every gain is attributable to the library. This document states the idea, the implementation on SmolVLA in LIBERO and LIBERO-PRO, the first measurements and the plan.

1 Idea

Two lines of work meet here. *Code-as-policies* systems (including our own ASPIRE) let a language model write the robot's controller; they adapt fast but replace the learned policy with hand-written perception and motion code. *Activation steering* shows that adding a direction to a network's hidden states can change its behaviour without training; for VLAs this has been demonstrated with fixed, hand-picked directions. What is missing is a system that *discovers* which interventions help, *when* to apply them, and keeps that knowledge as reusable code.

Question. Can a coding agent autonomously discover, verify and compose programs that control the internal behaviour of a frozen VLA, so that the robot acquires new capabilities continually, without fine-tuning?

Why it could work. A fine-tuned VLA already contains most of the skills a perturbed task needs (grasp, place, open); failures under distribution shift (moved objects, paraphrased instructions) often look like the right skill applied to the wrong target, or a commitment that never happens. Such failures may be correctable by small changes of internal state that a program can trigger at the right moment, and the simulator gives cheap, exact feedback on whether a change helped (an episode takes 2–10 s here).

2 Formulation

Let π_θ be the frozen VLA. At each replanning step it maps the observation o_t (two images, proprioception) and the instruction ℓ to an action chunk $a_{t:t+H}$. Write h_l for the hidden state that layer l

adds to the residual stream. Three families of interventions, from cheapest to most expensive:

$$\text{activation:} \quad h'_l = h_l + \alpha(s)\, v_l, \qquad \|v_l\| = 1, \tag{1}$$

$$\text{parameter:} \quad W' = W + \textstyle\sum_i g_i(s)\, B_i A_i \quad \text{(low-rank adapters)}, \tag{2}$$

$$\text{action:} \quad a'_t = a_t + \Delta a(s_t). \tag{3}$$

Here s is everything a program may read: the observation, the episode so far, and *the VLA's internals* (Section 3.3). The first version keeps θ frozen and uses only (1) and simple action edits, so every effect is attributable to the library.

Library entry. An entry is executable knowledge $\mathcal{S}_i = (p_i, f_i, q_i)$: an applicability condition $p_i(s)$, a steering program $f_i(s)$ that returns interventions, and evidence q_i (success with and without the entry, on which tasks and initial states, at what simulator cost). A program can call other entries, so entries compose.

Closed-loop internal steering. Because p_i and f_i may read internal signals, a program can say for example: *if the action tokens attend to the wrist image less than usual and the projection on the "grasp committed" direction is low, push that direction in the middle VLM layers.* The agent writes such rules at the level of detail of assembly code, and the simulator decides whether they help.

3 Implementation

3.1 Policy and simulator

SmolVLA (450M parameters, checkpoint `lerobot/smolvla_libero`, fine-tuned on the four LIBERO suites). It has two stacks: 16 layers of a SmolVLM2 text model (width 960) that encode a *prefix* of agentview-image, wrist-image, instruction and state tokens once per chunk, and 16 layers of an *action expert* (width 720) that denoise a chunk of $H = 50$ actions in 10 flow-matching steps, attending to the prefix (cross-attention, with self-attention every second layer). The rollout copies lerobot's LIBERO evaluation exactly: 256×256 images rotated by 180°, state = end-effector position, axis-angle and gripper (8 numbers), initial state e mod 50 for episode e, 10 settling steps, the suite's step limit. Environments come from LIBERO-PRO, so the same code runs the trained suites and their perturbed versions (object positions swapped, paraphrased instructions, new objects).

3.2 Intervention sites

SmolVLA runs its layers in a custom loop, so whole-layer hooks never fire; each layer's MLP is still called as a module and its output is added to the residual stream. A forward hook on `layer.mlp` therefore implements (1) exactly. VLM-layer interventions act once per chunk on the prefix (and through the cached keys and values on every denoising step); expert-layer interventions act on every denoising step. Strengths are expressed in units of the spread of activations along the direction ("a $k\sigma$ push").

3.3 Introspection

Per chunk, a small summary that programs read and traces store:

- **attention**: for each expert layer, the share of the action tokens' attention on each prefix segment (agentview, wrist, instruction, state) and on the action tokens, averaged over heads and denoising steps;

- **mlp norms**: norm of each layer's mean MLP output;
- **projections**: projection of each layer's mean MLP output on named directions from the library;
- **behaviour**: end-effector trajectory, gripper and executed actions (every fifth step).

3.4 Directions

Directions are contrasts of recorded activations (mean over tokens, per layer): *success* — successful minus failed episodes of a task; *shift* — a trained task minus its perturbed version ("pull the representation back towards the training distribution"); later, instruction pairs and directions found by the agent itself.

3.5 Programs and library

Programs are deliberately *open*: the agent can log anything and intervene with any inference-time method. A program file may define four optional functions, and the context exposes the whole model:

- `setup(ctx)` once per episode: register hooks on any module (`ctx.hook`), patch methods such as the attention function (`ctx.patch`), rewrite the instruction (`ctx.instruction`);
- `program(t, obs, internals, ctx)` before every action chunk: return additive steering terms;
- `on_step(t, obs, action, ctx)` every control step: replace the action;
- `on_end(result, ctx)`: undo in-place edits.

`ctx.vla`, `ctx.vlm_layers` and `ctx.expert_layers` give the model, `ctx.log(key, value)` writes arbitrary values into the episode trace, `ctx.memory` keeps state within the episode. The only hard constraints: no simulator ground truth (object poses, simulator state), and the frozen weights must be identical after each episode (checked by the evaluator; adapters are allowed as separate modules loaded and removed by the program).

```
DIRECTIONS = ["goal_lan_0_shift_vlm"] # projected into ‘internals‘

def setup(ctx): # log per-token norms of VLM layer 8 on every prefix pass
    ctx.hook(ctx.vlm_layers[8].mlp,
             lambda m, i, out: ctx.log("vlm8_norms", out[0, :8].norm(dim=-1)))

def program(t, obs, internals, ctx): # pull the paraphrase towards the trained instruction
     until the grasp
    if obs["robot0_gripper_qpos"][0] < 0.03:
        return []
    return [ctx.push("goal_lan_0_shift_vlm", layer=7, sigmas=2.0)]

def on_step(t, obs, action, ctx): # slow down once something is held
    if obs["robot0_gripper_qpos"][0] < 0.03:
        action = action.copy(); action[:3] *= 0.5
        return action
```

`scripts/run_program.py` evaluates a program on development initial states (episodes 0–19) and writes traces; held-out initial states (20–49) are only used by `scripts/verify.py`, which promotes a program into the library when it beats the frozen VLA there (one-sided Fisher exact test, 5%).

3.6 Agent loop

1. Pick a failing or unreliable task; run the frozen VLA with traces.
2. Diagnose from behaviour and internals (where does the policy look, what does it commit to, where does it diverge from a trained-scene rollout).
3. Propose interventions: build directions, choose sites, write a program with conditions; evaluate on development episodes; iterate within a budget.
4. Verify the best program on held-out initial states; store it only if it beats the frozen VLA there.
5. Reuse: on a new task, first try the library's entries (and compositions) before searching.

Escalation order: logging only → activation and attention edits → action edits → short low-rank adapter runs (at most one at a time, minutes of GPU), with every entry recording its simulator cost so methods can be compared at equal budget. A verified program can later be *compiled* into an adapter by distilling its behaviour.

4 First measurements

suite	0	1	2	3	4	5	6	7	8	9	mean
`libero_goal`	1.0	1.0	1.0	0.5	0.9	0.6	0.4	1.0	1.0	1.0	0.84
`libero_goal_lan`	0.2	0.7	0.7	0.0	0.8	0.0	0.2	0.1	0.1	0.5	0.33
`libero_goal_swap`	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.00
`libero_spatial`	1.0										1.00

Table 1: Frozen SmolVLA (`lerobot/smolvla_libero`): success rate per task (10 episodes, initial states 0–9). `_swap`: object positions swapped; `_lan`: paraphrased instruction.

No steering sweep yet.

5 Plan

1. **Baseline** (done for the suites above): frozen SmolVLA per task, 10 episodes.
2. **Steering works at all**: record mixed-outcome and trained/perturbed pairs, build success and shift directions, sweep layer × strength on held-out initial states. Success criterion: an intervention that raises success on held-out states of a task where the frozen policy fails, by more than the binomial noise (with 30 episodes, a rise from 0.2 to 0.5 is significant at the 5% level).
3. **State-dependent steering**: the same directions gated by phase (before/after grasp) and by internal signals.
4. **Agent-written programs**: the coding agent runs the loop above on LIBERO-PRO perturbed tasks; library entries verified on held-out states.
5. **Transfer**: entries discovered on some perturbed tasks evaluated on other perturbed tasks and suites, without search; compare with search from scratch at equal simulator budget.
6. **Parameter level**: short low-rank adapter runs on self-generated successful rollouts, and compilation of verified programs into adapters.

Baselines and ablations. Frozen VLA; fixed steering (no state dependence); state-dependent steering without internal signals; programs without the library (search per task); full library. An equal-budget comparison against learned residual corrections or adapters trained with the same number of simulator episodes is needed to show that the benefit comes from program synthesis and not from simulator experience alone.

Risks. (i) Steering directions may not be causal or may not exist for a failure mode; the sweep in step 2 tests this before any agent work. (ii) Gains on development initial states may be noise; verification on held-out states and binomial confidence intervals guard against it. (iii) The perturbed suites may be failures of perception that no internal push can fix; language perturbations are the most likely to respond to steering, position swaps the least.

6 Related work (to be checked in detail)

Mechanistic steering of VLAs with fixed internal directions; synthesised steering objectives for frozen robot policies; learned latent steering triggered when failure is likely; libraries of task-specific adapters with dynamic routing; residual corrections trained in simulation for frozen VLAs; and code-as-policies systems with skill libraries (ASPIRE). The distinctive claim here is the automated synthesis, verification, composition and reuse of state- and internal-signal-dependent steering programs; a full literature check is needed before stating novelty.